Ejercicio #1

```
public class Box<T> {
    private T value

    public void save(T value){
        this.value = value;
    }

    public T get() {
        return value;
    }
}


public class Main {
    public static void main(String[] args) {
        Box<String> box = new Box<>();
        box.save("Hola");
        System.out.println(box.get());

        Box<Integer> box2 = new Box<>();
        box2.save(1);
        System.out.println(box2.get());
    }
}
```

Preguntas #1

1. El código se repite mucho; es lo mismo, solo cambia el tipo de dato.

2. Tendríamos que duplicar aún más código.

3. Porque hay que agregar codigo nuevo constantemente

1. Porque T permite usar cualquier tipo.

2. En tiempo de compilación.

3. Porque detectan los errores de tipo antes de ejecutar.

Ejercicio #2

```
public class IOManager {
    public static <T> void print(T value) {
        System.out.println(value);
    }
}
```

Preguntas #2

1. Indica que el método es genérico.

2. Sí, porque el compilador verifica los tipos antes de compilar.

3. Collections.sort(List<T> list);
   Collections.max(Collection<T> coll);

Ejercicio #3

```
interface Attackable<T> {
        void attack(T objective);
}

public class Enemy implements Attackable<Player>{
    private String name;
    public Enemy(String name){
        this.name=name;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    @Override
    public void attack(Player objective) {
        System.out.println("Enemy attacks player");
    }
    public String toString() {
        return "Enemy:" + name;
}
```

```
public class Player implements Attackable<Enemy>{
    private String name;

    public Player(String name){
        this.name = name;
    }

    public String getName(){
        return name;
    }

    public void setName(String name){
        this.name = name;
    }

    @Override
    public void attack(Enemy objective){
        System.out.println("Player attacks enemy
    }

    public String toString(){
        return "Player" + name;
    }
}
```

```
public class Main {
    public static void main (String[] args) {
        Player player = new Player ("Hero");
        Enemy enemy = new Enemy ("Goblin");

        player.attack (enemy);
        enemy.attack (player);

        System.out.println(player.toString());
        System.out.println(enemy.toString());
    }
}
```

Preguntas #3

1. Indica el tipo de parametro que usará la interfaz

Ejercicio #4

1. Va a mostrar un error directamente en tiempo de compilación, no hace ni falta ejecutar el programa.